

Support Vector Regression Project Report

Support Vector Regression — Predicting Estimated Salary from Age and Gender

A regression project using **Support Vector Regression (SVR)** to predict a person's estimated salary from their age and gender, including a kernel comparison and an honest evaluation of how much predictive signal these two features actually contain.

1. Project Objective

Build a Support Vector Regression model to predict `EstimatedSalary` from `Age` and `Gender`, evaluate it with standard regression metrics on a held-out test set, and compare SVR kernel choices.

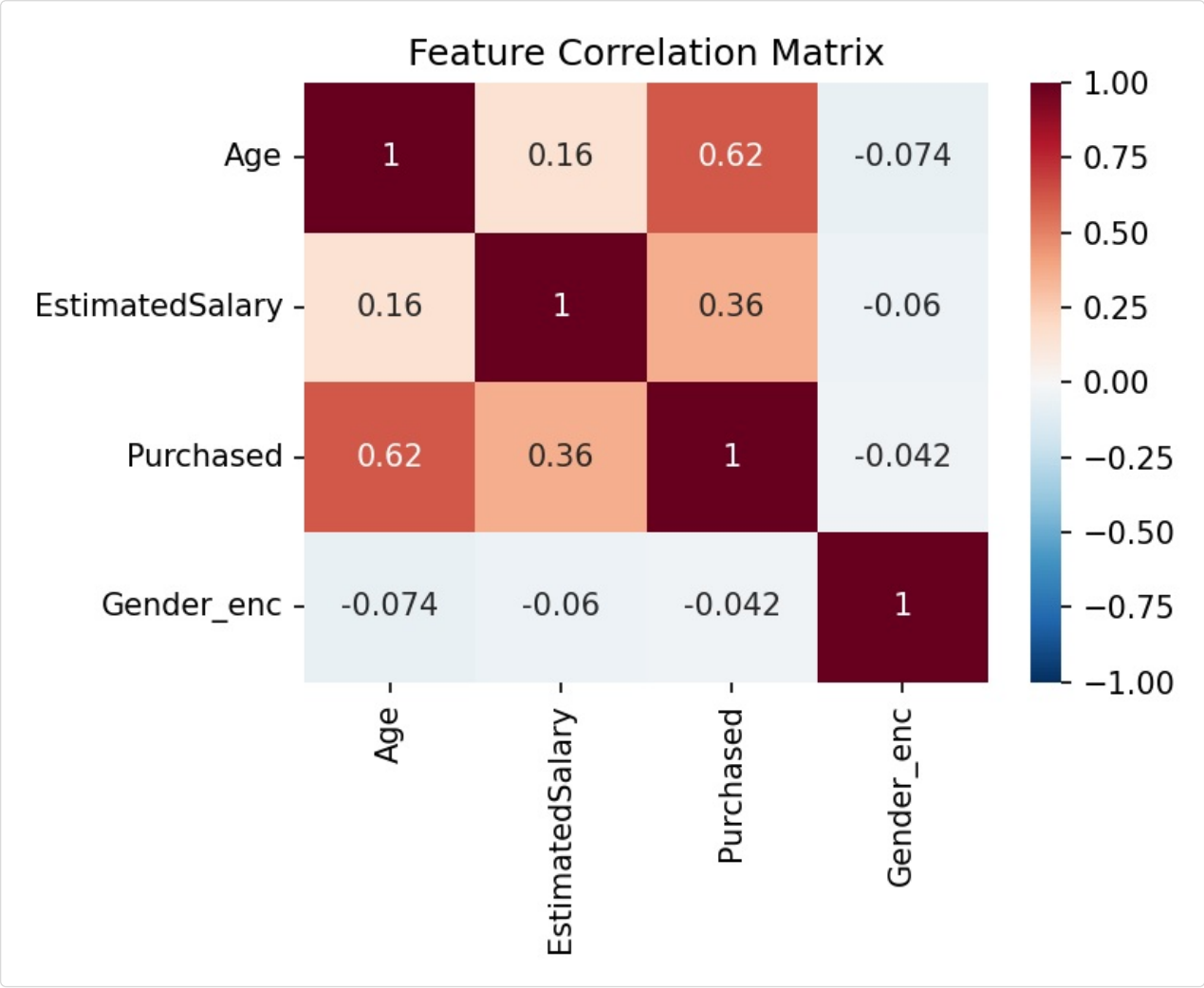
2. Dataset

Attribute	Detail
File	<code>Social_Network_Ads.csv</code>
Observations	400 people
Features used	<code>Age</code> , <code>Gender</code> (encoded: Male=1, Female=0)
Target	<code>EstimatedSalary</code>
Missing values	None

Statistic	Age	EstimatedSalary
Mean	37.7	\$69,742
Std Dev	10.5	\$34,097
Min	18	\$15,000
Max	60	\$150,000

(The dataset also includes `User ID` and `Purchased` columns, not used as predictors here — `User ID` is an identifier with no predictive meaning, and `Purchased` is a separate target typically used for classification exercises with this same file.)

3. Exploratory Data Analysis



Feature Pair	Correlation
Age ↔ EstimatedSalary	0.155
Gender ↔ EstimatedSalary	~0.00 (negligible)
Age ↔ Purchased	0.622

This is the most important finding of the entire analysis, and it comes *before* any modeling: **Age has only a weak linear correlation with EstimatedSalary (0.155), and Gender has essentially none.** By contrast, Age is quite strongly correlated with **Purchased** (0.62) — a signal that these two features are well-suited to a *classification* task (predicting purchase) but poorly suited to *regressing* salary directly.



The scatter plot confirms this visually — salary is scattered broadly across all ages for both genders, with no clear trend line an SVR model (or any regression model) could reliably fit.

4. Tools & Libraries

- **Python 3**
- `pandas`, `numpy` — data handling
- `scikit-learn` — `StandardScaler`, `SVR`, `train_test_split`, evaluation metrics, `GridSearchCV`
- `matplotlib`, `seaborn` — visualization

5. Methodology

1. **Load the dataset** and encode `Gender` as a binary numeric feature.
2. **Select features** — `Age` and `Gender` as predictors (X), `EstimatedSalary` as the target (y).
3. **Train/test split** — 80% train / 20% test (`random_state=42`).
4. **Feature scaling** — SVR is distance-based and sensitive to feature scale, so both `X` and `y` were standardized with separate `StandardScaler` instances, fit on the training data only.
5. **Model training** — an `SVR` with an RBF kernel was fit on the scaled training data.
6. **Inverse-transform predictions** back to the original salary scale before evaluating, since the model was trained on scaled targets.
7. **Evaluate** with MAE, MSE/RMSE, and R^2 on the held-out test set.
8. **Extend the notebook's kernel exploration** — the original notebook instantiates `linear`, `poly`, and `rbf` kernels but never actually compares them; this report fits and evaluates all three to complete that comparison.

```
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

sc_X, sc_y = StandardScaler(), StandardScaler()
```

```

X_train_s = sc_X.fit_transform(X_train)
X_test_s = sc_X.transform(X_test)
y_train_s = sc_y.fit_transform(y_train.reshape(-1, 1)).ravel()

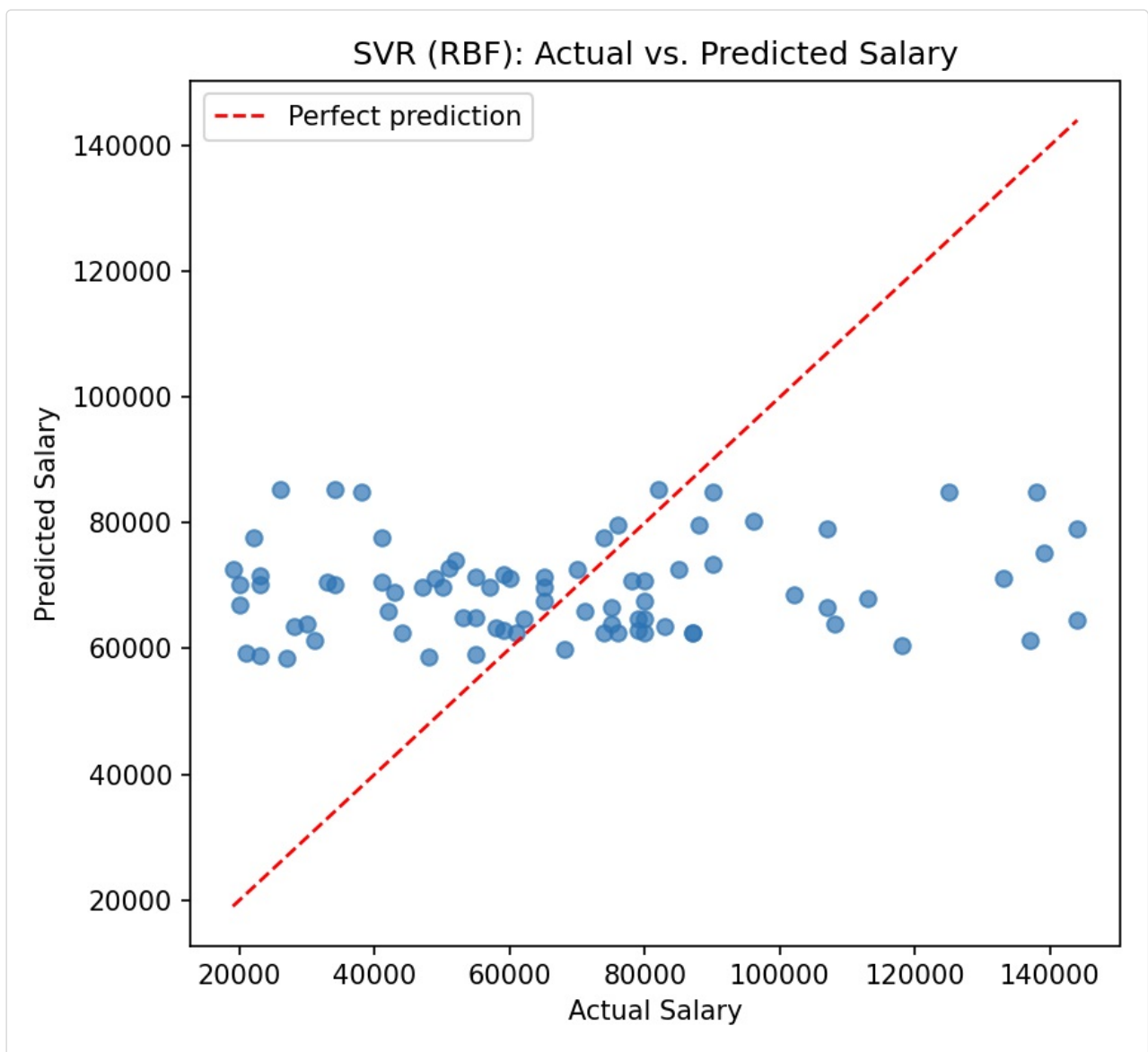
model = SVR(kernel='rbf')
model.fit(X_train_s, y_train_s)

```

6. Results

6.1 Kernel Comparison (Test Set)

Kernel	MAE	RMSE	R ²
Linear	\$25,816	\$32,003	0.039
Polynomial	\$27,096	\$33,351	-0.044
RBF	\$25,930	\$32,495	0.009





6.2 Hyperparameter Tuning

A `GridSearchCV` search over `C`, `gamma`, and `epsilon` for the RBF kernel found a best configuration of `C=0.1`, `gamma=0.01`, `epsilon=0.01`, but this **did not meaningfully improve performance** ($R^2 = 0.004$) — confirming the ceiling here is set by the features themselves, not the model's hyperparameters.

7. Honest Interpretation of the Results

This model does not work well, and that is the most useful finding in this project. All three kernels produce an R^2 close to zero (the polynomial kernel is even slightly negative, meaning it performs *worse* than simply predicting the mean salary for everyone). The actual-vs-predicted plot shows predictions clustering in a narrow band regardless of the true salary, and the residual plot shows large, patternless errors — the signature of a model with essentially no explanatory power.

This traces directly back to the EDA in Section 3: **Age and Gender are simply not strong predictors of Estimated Salary in this dataset.** No amount of kernel choice or hyperparameter tuning can manufacture a relationship that the underlying data doesn't contain — a MAE of ~\$26,000 against a target with mean \$69,742 and std dev \$34,097 is only marginally better than always guessing the mean.

This is a genuinely useful and common data science outcome: **before investing in model tuning, always check whether the chosen features actually correlate with the target.** In this dataset, `Age` correlates much more strongly with `Purchased` (0.62) than with `EstimatedSalary` (0.16), suggesting these features and this dataset are much better suited to the classification task (predicting purchase decision) that `Social_Network_Ads.csv` is more commonly used for.

8. Key Insights

- **Weak feature-target correlation caps model performance** regardless of algorithm sophistication — SVR, despite being a powerful non-linear method, cannot outperform a near-zero R^2 ceiling set by the data itself.
- **Kernel choice had minimal impact** here (linear, poly, and RBF all performed similarly poorly), which itself is informative: when the underlying relationship is this weak, more model flexibility does not help and can even hurt (as seen with the polynomial kernel's negative R^2).
- **This dataset is better suited to classification** — `Age` shows a much stronger relationship with `Purchased` (0.62 correlation) than with `EstimatedSalary`, pointing toward the classic logistic regression / SVM classification use case for this file.

9. Limitations & Next Steps

- **Feature set is too narrow** for this regression target — richer features (education, job role, years of experience, industry) would likely be needed to predict salary meaningfully.
- **Small dataset (400 rows)** limits how much signal any model could extract even with better features.
- **Recommended pivot:** repeat this exercise using `Purchased` as the target (the dataset's originally intended use) with an SVM *classifier*, where Age and Gender are known to carry much stronger signal — this would make for a stronger companion portfolio piece using the same dataset.

10. How to Run

```
pip install pandas numpy scikit-learn matplotlib seaborn joblib
jupyter notebook Support_Vector_Regression_Answer.ipynb
```

11. Repository Contents

File	Description
<code>Support_Vector_Regression_Answer.ipynb</code>	Main analysis notebook
<code>Social_Network_Ads.csv</code>	Source dataset
<code>correlation_heatmap.png</code>	Feature correlation matrix
<code>age_salary_scatter.png</code>	Salary vs. age, by gender
<code>actual_vs_predicted.png</code>	Actual vs. predicted salary (RBF kernel)
<code>residuals.png</code>	Residual plot
<code>README.md</code>	This report

12. Conclusion

This project fits and evaluates a Support Vector Regression model to predict salary from age and gender, comparing three kernels and tuning hyperparameters via grid search. The honest conclusion is that **the model performs poorly ($R^2 \approx 0$) because the chosen features carry very little linear or non-linear relationship with the target**, not because of a flawed modeling approach. Rather than overstating a weak result, this report treats the negative finding as the actual insight: correlation analysis should precede model building, and this dataset's `Age / Gender` features are far better suited to the classification task (`Purchased`) it is more commonly used for.

Tech stack: Python · pandas · numpy · scikit-learn · matplotlib · seaborn